Tema 08

E/S en Archivos y Dispositivos

Programación en Lenguaje C

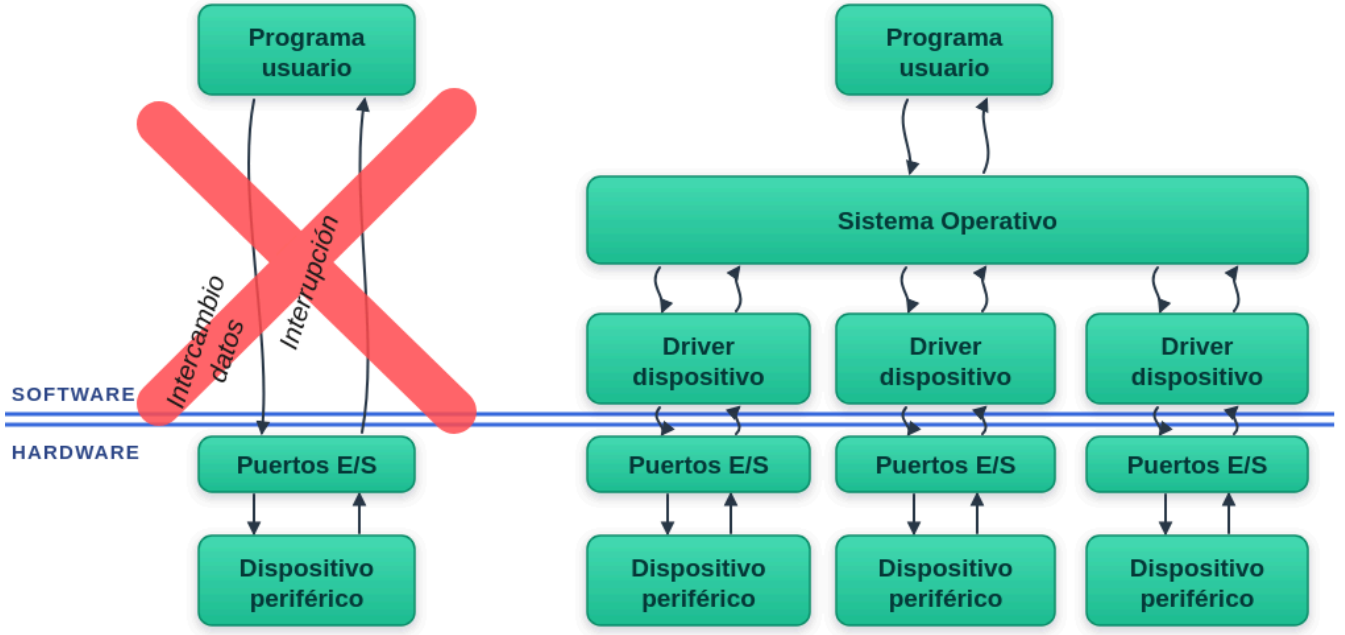
Ingeniería de Sistemas y Automática - Universidad de Oviedo

Índice del Tema

1. Concepto de Entrada/Salida por *Streams* y arquitectura SO/Driver
2. Tipos de Streams y denominación de dispositivos
3. Apertura y cierre de streams (`fopen`, `fclose`, tipo `FILE*`)
4. Entrada/Salida orientada a texto (`fgetc`, `fgets`, `fprintf`, `fscanf`)
5. Streams estándar de consola (`stdin`, `stdout`, `stderr`) y redirección
6. Entrada/Salida binaria (`fread`, `fwrite`)
7. Posicionamiento y control de errores (`fseek`, `ftell`, `feof`, `fflush`)
8. E/S de bajo nivel sin buffer (POSIX / Unix)

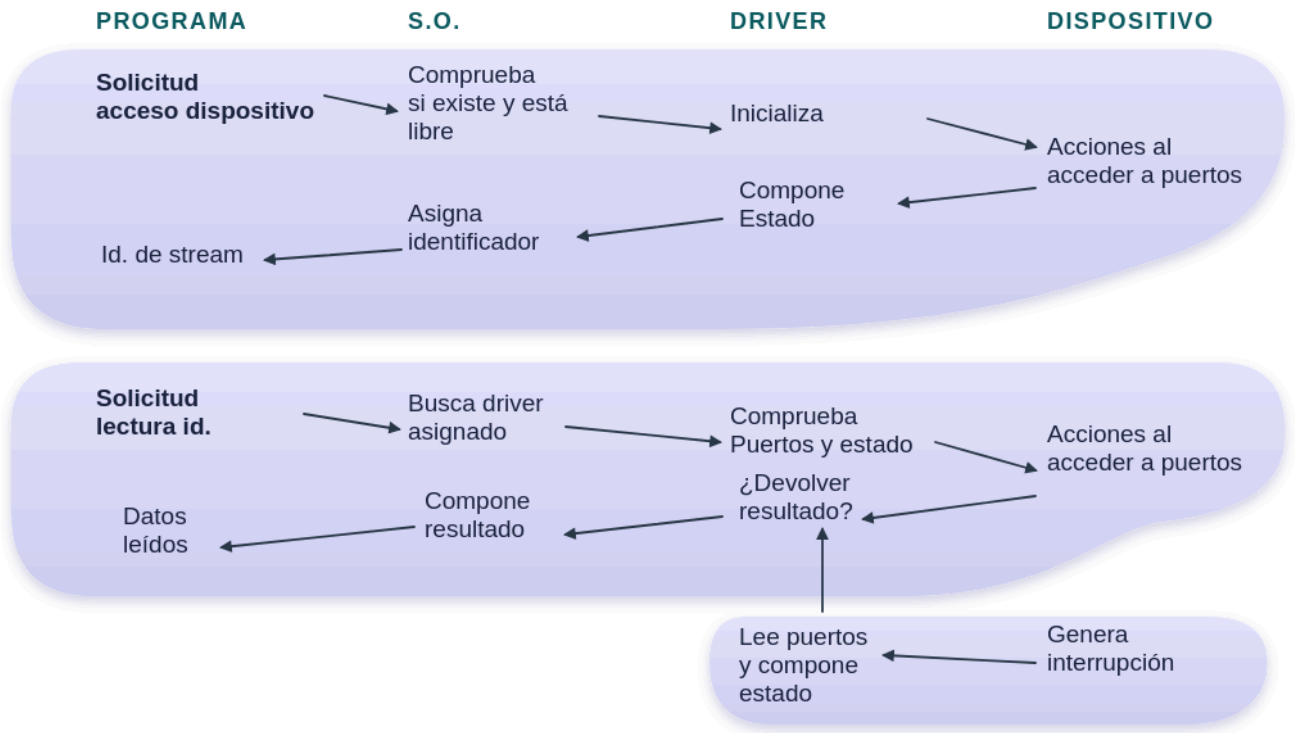
E/S por Stream

Permite a los programas intercambiar información con dispositivos periféricos mediante streams:



E/S por Stream

Funcionamiento de la E/S por stream:



E/S por Stream

Un **Stream** es una secuencia ordenada de bytes enviada a o recibida desde un dispositivo. Todos los dispositivos se tratan de forma homogénea.

Tipos de Streams:

Según dirección:

- Entrada
- Salida
- Entrada/Salida (bidireccional)

Según soporte:

- **Con memoria:** Archivos en disco
- **Sin memoria:** Teclado, pantalla, puerto serie...

Según formato:

- Orientados a texto
- Binarios

Denominación e Identificación de Dispositivos:

El Sistema Operativo asigna un nombre lógico mediante una **cadena de caracteres** para mapear tanto archivos como hardware físico:

- **Puertos serie / Hardware:** `"COM1"` (Windows) o `"/dev/ttyS0"` (Linux/POSIX).
- **Archivos en almacenamiento:** Rutas absolutas/relativas como `"C:\\datos.txt"` o `"/home/user/log.txt"`.

- El **Sistema Operativo** se encarga de la organización lógica (nombres, drivers, permisos, estado).
- Los **Drivers** se encargan del acceso físico (puertos, gestión de interrupciones).

Funciones para manejo de streams

- Permiten asignar un identificador a un stream, leer/escribir y liberar el recurso.
- Requieren incluir la librería `<stdio.h>`.
- El identificador de stream se declara como una variable de tipo `FILE*`.

- **Funciones de apertura / cierre de stream:**

```
FILE* fopen(const char* name, const char* mode);
```

- **name:** Nombre o ruta del dispositivo (ej. puerto serie `"COM1"`) o archivo de almacenamiento (ej. `"datos.txt"`).
- **mode:** Modo de apertura: `"r"` (lectura), `"w"` (escritura/sobrescritura), `"a"` (añadir). Añadir `"b"` para binario (ej. `"rb"`).

```
int fclose(FILE* fid);
```

- Cierra el stream, libera recursos y fuerza el volcado de los búferes al soporte físico.

⚠ **Comprobación obligatoria: Si `fopen` falla o no encuentra el recurso, devuelve `NULL`. Siempre se debe validar antes de operar sobre el stream.**

Ejemplo Práctico en C (Apertura y Cierre)

- **Patrón estándar de manipulación segura de archivos:**

```c
#include <stdio.h>

int main(void) {
    // 1. Declaración del identificador del stream
    FILE *fid;

    // 2. Apertura del archivo en modo escritura texto ("w")
    fid = fopen("datos.txt", "w");

    // 3. Comprobación obligatoria de puntero NULL
    if (fid == NULL) {
        printf("Error: No se pudo abrir el archivo.\n");
        return 1;
    }

    // 4. Operación sobre el fichero
    fprintf(fid, "Sensor 1: %f\n", 23.5f);

    // 5. Cierre obligado para asegurar el volcado físico
    fclose(fid);

    return 0;
}
```

E/S Orientada a Texto

▫ Carácter a carácter:

`int fgetc(FILE* fid);`
- ▸ **Qué hace:** Lee 1 carácter.
- ▸ **Devuelve:** Código ASCII (`int`) o **EOF (-1)** si falla/fin.

`int fputc(int car, FILE* fid);`
- ▸ **Qué hace:** Escribe 1 carácter.
- ▸ **Devuelve:** El carácter escrito o **EOF (-1)** en error.

Nota: Devuelven `int` para representar **EOF (-1)**, valor fuera del rango de `char` (0 a 255).

▫ Línea a línea (Cadenas):

`char* fgets(char* str, int n, FILE* fid);`
- ▸ **Qué hace:** Lee 1 línea (máx *n-1* chars + `'\0'`).
- ▸ **Devuelve:** Puntero **`str`** o **NULL** si hay error/EOF.

`int fputs(const char* str, FILE* fid);`
- ▸ **Qué hace:** Escribe una cadena de texto.
- ▸ **Devuelve:** Entero ≥ 0 o **EOF (-1)** en caso de error.

Nota: `str` pasa el búfer donde guardar; el retorno devuelve esa misma dirección para encadenar comprobaciones.

▫ Con formato:

`int fscanf(FILE* fid, const char* fmt, ...);`
- ▸ **Qué hace:** Lee y parsea datos formateados.
- ▸ **Devuelve:** Nº de datos asignados o **EOF** si falla el 1º.

`int fprintf(FILE* fid, const char* fmt, ...);`
- ▸ **Qué hace:** Escribe texto formateado.
- ▸ **Devuelve:** Nº de chars escritos o **< 0** si hay error.

¿Qué es EOF? Es una constante predefinida en `<stdio.h>` (abreviatura de *End Of File*, equivalente a `-1`) devuelta para indicar el final del archivo o un error.

Ejemplo Práctico: E/S de Texto

- **Escritura formateada (`fprintf`) y lectura línea a línea (`fgets`) en archivo de texto:**

```c
#include <stdio.h>

int main(void) {
    FILE *fid = fopen("medidas.txt", "w"); // Modo escritura texto
    if (fid == NULL) return 1; // Comprobacion de error de apertura

    // 1. Escritura formateada en texto
    fprintf(fid, "Sensor_01 %f\n", 24.5f);
    fclose(fid); // Cierra y vacia el bufer al disco

    // 2. Lectura linea a linea desde texto
    fid = fopen("medidas.txt", "r"); // Modo lectura texto
    if (fid != NULL) {
        char buffer[50];
        while (fgets(buffer, 50, fid) != NULL) {
            printf("Leido: %s", buffer); // Muestra la linea en consola
        }
        fclose(fid);
    }
    return 0;
}
```

Puntos clave: Usamos modos `"w"` y `"r"` para texto. `fgets` previene desbordamientos de memoria al limitar explícitamente la cantidad máxima de caracteres a leer (en este caso `50`) y devuelve `NULL` cuando alcanza el fin de archivo (EOF).

Streams Estándar de Consola

▫ **Streams por defecto: permiten realizar E/S a consola con `printf()`, `scanf()`, `gets()`, `puts()`, `getchar()`, `putchar()`:**

`stdin`
- ▶ **Entrada estándar**
- ▶ Dispositivo: Teclado.
- ▶ Modo: Lectura en texto.

`stdout`
- ▶ **Salida estándar**
- ▶ Dispositivo: Pantalla/Monitor.
- ▶ Modo: Escritura en texto.

`stderr`
- ▶ **Error estándar**
- ▶ Dispositivo: Pantalla (errores).
- ▶ Modo: Escritura sin búfer.

▫ **Equivalencias con las funciones generales de Stream:**

```
printf(...) ≡ fprintf(stdout, ...)   scanf(...) ≡ fscanf(stdin, ...)
putchar(c) ≡ fputc(c, stdout)        getchar() ≡ fgetc(stdin)
```

▫ **Redirección desde el Sistema Operativo (Windows / Linux):**

El Sistema Operativo permite cambiar la fuente u origen de `stdin` y `stdout` sin necesidad de modificar ni recompilar el código en C:

```
Miprograma.exe < entrada.txt > salida.txt
```
(< redirige stdin | > redirige stdout)

Funciones para manejo de streams

Funciones de E/S en streams binarios:

Lectura de datos:

```
int fread(void* buffer, size_t size, size_t count, FILE
*fid);
```

- Lee un conjunto de count*size bytes del stream y los almacena en el mismo formato en el buffer

Escritura de datos:

```
int fwrite(const void* buffer, size_t size, size_t count,
FILE *fid);
```

- Envía al stream un conjunto de count*size bytes indicados a partir de la dirección de memoria buffer

En ambos casos:

- Las funciones devuelven el nº de elementos (bytes/size) realmente escritos o leídos. Si es < `count`, indica EOF o error.

Ejemplo Práctico: E/S Binaria

- **Escritura (`fwrite`) y lectura (`fread`) directa de memoria RAM a disco:**

```c
#include <stdio.h>

int main(void) {
    float datos_out[3] = {12.5f, 24.0f, 18.2f};
    float datos_in[3];

    // 1. ESCRITURA BINARIA ("wb")
    FILE *fid = fopen("sensor.bin", "wb");
    if (fid != NULL) {
        // Escribe los 3 floats (3 x 4 bytes = 12 bytes crudos)
        size_t escritos = fwrite(datos_out, sizeof(float), 3, fid);
        printf("Elementos escritos: %zu\n", escritos); // Devuelve 3
        fclose(fid);
    }

    // 2. LECTURA BINARIA ("rb")
    fid = fopen("sensor.bin", "rb");
    if (fid != NULL) {
        // Lee 3 floats y los deposita directamente en datos_in
        size_t leidos = fread(datos_in, sizeof(float), 3, fid);
        printf("Elementos leidos: %zu\n", leidos);     // Devuelve 3
        fclose(fid);
    }
    return 0;
}
```

Aspectos clave: Usamos modos binarios (`"wb"` y `"rb"`). `sizeof(float)` le indica el tamaño del elemento individual (4 bytes) y `3` el número de elementos. Al ser E/S en bloque, los bytes pasan directo entre el array de la RAM y el disco sin formatear a texto ASCII.

Funciones para manejo de streams

- **Funciones auxiliares para streams con almacenamiento (archivos):**

```
int feof(FILE* fid);
```

▶ Indica si se ha alcanzado el fin de archivo

```
void rewind(FILE* fid);
```

▶ Vuelve al principio del archivo

```
long ftell(FILE *fid);
```

▶ Devuelve la posición actual (en bytes) desde el principio del stream

```
int fseek(FILE *fid, long offset, int origin);
```

▶ Coloca el puntero en la posición (bytes) deseada a partir de:

- el principio: **origin = SEEK_SET**
- la posición actual: **origin = SEEK_CUR**
- el final: **origin = SEEK_END**

Funciones para manejo de streams

- **Funciones auxiliares para streams:**

```
int ferror(FILE* fid);
```

► Devuelve el código del último error (0 = ok)

```
void clearerr(FILE* fid);
```

► Borra el código del último error

```
int fflush(FILE *fid);
```

► Vacía los búferes intermedios con los que el S.O. gestiona la E/S física del dispositivo asociado al stream:

- Si estaba abierto para salida, asegura que dicha salida se escribe físicamente.
- Si estaba abierto para entrada, elimina los datos de los búferes intermedios.

Resumen del Tema

1. El modelo de Streams en C abstrae el intercambio de información con cualquier periférico o archivo.
2. `fopen` permite seleccionar modos de apertura en texto o binario.
3. Para ficheros formateados usar `fprintf`/`fscanf`; para grandes volúmenes de datos crudos usar `fwrite`/`fread`.
4. Comprobar siempre que `fopen` no devuelva `NULL` antes de operar sobre un stream.
5. Utilizar `fflush` cuando sea crítico asegurar la escritura inmediata sobre hardware.